

Formal Verification of Rust Programs via Embedded Monadic Languages and Agentic Workflows in Lean 4

Feasibility and Structural Alignment

The formal verification of production-grade software written in Rust using the Lean 4 theorem prover is not only highly feasible, but it is also exceptionally well-aligned with the programming language's core semantics.¹ Traditional systems programming languages, such as C and C++, introduce unrestricted pointer aliasing and manual memory management, forcing verification frameworks to employ complex, stateful separation logics or elaborate heap models that impose heavy proof development costs.¹

Rust, by contrast, relies on a static type system based on a linear, substructural ownership and borrow-checking discipline.¹ The compiler enforces a fundamental invariant: any given value is either accessible via multiple immutable references (aliasing) or a single mutable reference (mutability), but never both simultaneously.³ This "aliasing XOR mutability" rule ensures that safe Rust programs are functional in essence, behaving as pure value-passing systems in which changes to mutable references propagate deterministically.⁴ Consequently, the challenge of program verification is transformed from reasoning about pointer arithmetic, memory layouts, and framing conditions into proving properties of pure, mathematical functions—a domain in which interactive proof assistants like Lean 4 excel.¹

To bridge the gap between Rust source code and Lean 4, a verifier operates on structured intermediate representations.⁷ The standard Rust compiler (rustc) desugars source code into Mid-level IR (MIR).⁷ While MIR represents borrow operations, drops, and moves explicitly, its control-flow graph is unstructured, containing arbitrary jumps and implicit panic unwinding handlers that hinder direct functional mapping.⁸ To resolve this, compiler-integrated frontends, such as Charon, process MIR into Unstructured Low-Level Borrow Calculus (ULLBC).⁷ ULLBC simplifies pattern matching and constants before reconstructing the control-flow graph into Low-Level Borrow Calculus (LLBC).⁷ LLBC recovers structured constructs, including loops, matches, and sequential conditional branches.⁷ The resulting structured abstract syntax tree (AST) can then be translated directly into purely functional models in Lean 4.¹

Designing the Formal Target Language in Lean 4

When constructing a Lean 4 library for Rust verification, the core architectural decision involves defining the formal target language that models the translated programs.¹ Developers can choose from three primary language representation paradigms.

Shallow Monadic Embedding with Dijkstra Monads

In a shallow monadic embedding, Rust programs are compiled directly into native Lean 4 monadic terms.¹² Rather than representing the syntax of the program as a data structure, the program's operations are modeled using a stack of monad transformers that encapsulate the necessary computational effects.¹²

To capture Rust's execution characteristics, a state-error monad stack is constructed.² This monad represents computations that can modify a local state (modeling variable updates) and can either succeed or crash (modeling panics or runtime assert violations).⁴ In Lean 4, this state-error monad, denoted as **Result α** , is defined as:

$$\text{Result } \alpha = \text{Ok } \alpha \mid \text{Panic}$$

By leveraging monad transformer algebras, developers can automatically derive verification condition generators (VCGs) and Dijkstra monads.¹² This architecture translates Rust's sequential bindings into monadic binds ($\gg=$), enabling linear composition of functional state updates while producing clean, mathematical proof obligations that can be discharged using automated or interactive tactics.¹

Deep Embedding via Syntactic ASTs and Big-Step Semantics

Alternatively, a formal language can be deeply embedded by defining the AST within Lean 4 using inductive datatypes.¹³ This deep embedding defines explicit datatypes for expressions (Expr), statements (Stmt), values (Value), and unary/binary operators.¹³ The operational semantics of the language are specified formally as an inductive proposition representing a relational big-step or small-step evaluation.¹³

To run these embedded programs, developers construct an executable interpreter in Lean 4.¹³ Because Rust loops can be non-terminating, the interpreter must be parameterized with a "fuel" variable (a decreasing natural number, Nat, that limits the maximum execution steps) to satisfy Lean's strict termination requirements.¹³ The verification library then proves that the executable interpreter is both sound and complete with respect to the relational big-step specification.¹³ This approach is highly robust and isolates the verification logic from the parser, but it requires verifying both the implementation and the interpreter.¹³

Custom Syntactic Domain-Specific Languages

Lean 4's extensible parser and macro system allow the construction of custom, syntactic domain-specific languages (DSLs) directly within the proof assistant.¹³ Using metaprogramming frameworks like DSLean, developers can establish bidirectional translations between external Rust intermediate representations (such as LLBC) and native Lean expressions.¹⁴

This DSL paradigm utilizes Lean's native elaboration, unification, and metavariable resolution to enforce type-correctness on the translated AST in real time.¹⁴ By defining syntactic macros—such as ``—verification engineers can write, visualize, and reason about Rust-like syntax directly within Lean, utilizing syntactic matching to resolve structures without relying on unsafe, meta-level transformations.¹³

Borrow Semantics and the Forward-Backward Function Paradigm

To model Rust's mutable references ($\&\text{mut } T$) in a purely functional, value-passing setting without introducing a complex heap model, the translation employs the *Forward-Backward* paradigm.³

Because mutable references allow temporary access to modify a value, their behavior is split into two sequential phases⁴:

1. **The Forward Function** (f_{fwd}): Models the standard execution of the Rust function up to its return point, producing the final returned value.⁴
2. **The Backward Functions** (f_{back}): For each lifetime parameter present in the function signature, a corresponding backward function is generated.⁴ When a borrow's lifetime ends, any modifications made through that borrow must propagate back to the original source variable.⁴ The backward function takes the original inputs alongside the updated value of the returned reference and returns a tuple containing the updated values of the input borrows.⁴

To illustrate this mathematically, consider a generic Rust function that mutably selects and returns one of two borrows based on a boolean condition:

Rust

```
fn choose<'a, T>(b: bool, x: &'a mut T, y: &'a mut T) -> &'a mut T {  
  if b { x } else { y }  
}
```

A direct translation is problematic because the caller can subsequently mutate the returned reference, which indirectly mutates either x or y depending on the runtime branch taken.⁴

Under the forward-backward paradigm, this is translated into two pure functions in Lean 4⁴:

$$\text{choose_fwd} : \text{Type} \rightarrow \text{Bool} \rightarrow T \rightarrow T \rightarrow \text{Result } T$$

$$\text{choose_back} : \text{Type} \rightarrow \text{Bool} \rightarrow T \rightarrow T \rightarrow T \rightarrow \text{Result } (T \times T)$$

The forward function returns the selected value ⁴:

$$\text{choose_fwd } \tau \ b \ x \ y = \text{if } b \text{ then Return } x \text{ else Return } y$$

The backward function tracks the termination of the borrow. ⁴ It receives the updated value of the returned reference as its final argument (representing the state of the reference when its lifetime ends) and propagates the update back to the inputs ⁴:

$$\text{choose_back } \tau \ b \ x \ y \ ret = \text{if } b \text{ then Return } (ret, y) \text{ else Return } (x, ret)$$

At the call site, consecutive mutations are chained using monadic let-bindings. ¹ For instance, if the caller modifies the returned reference, the new state is passed back into `choose_back`, which returns the updated states of `x` and `y`. ⁴ This mechanism successfully eliminates the need for pointer tracking, memory addresses, or frame rules, mapping Rust's low-level memory behavior directly to standard, pure functional equations. ¹

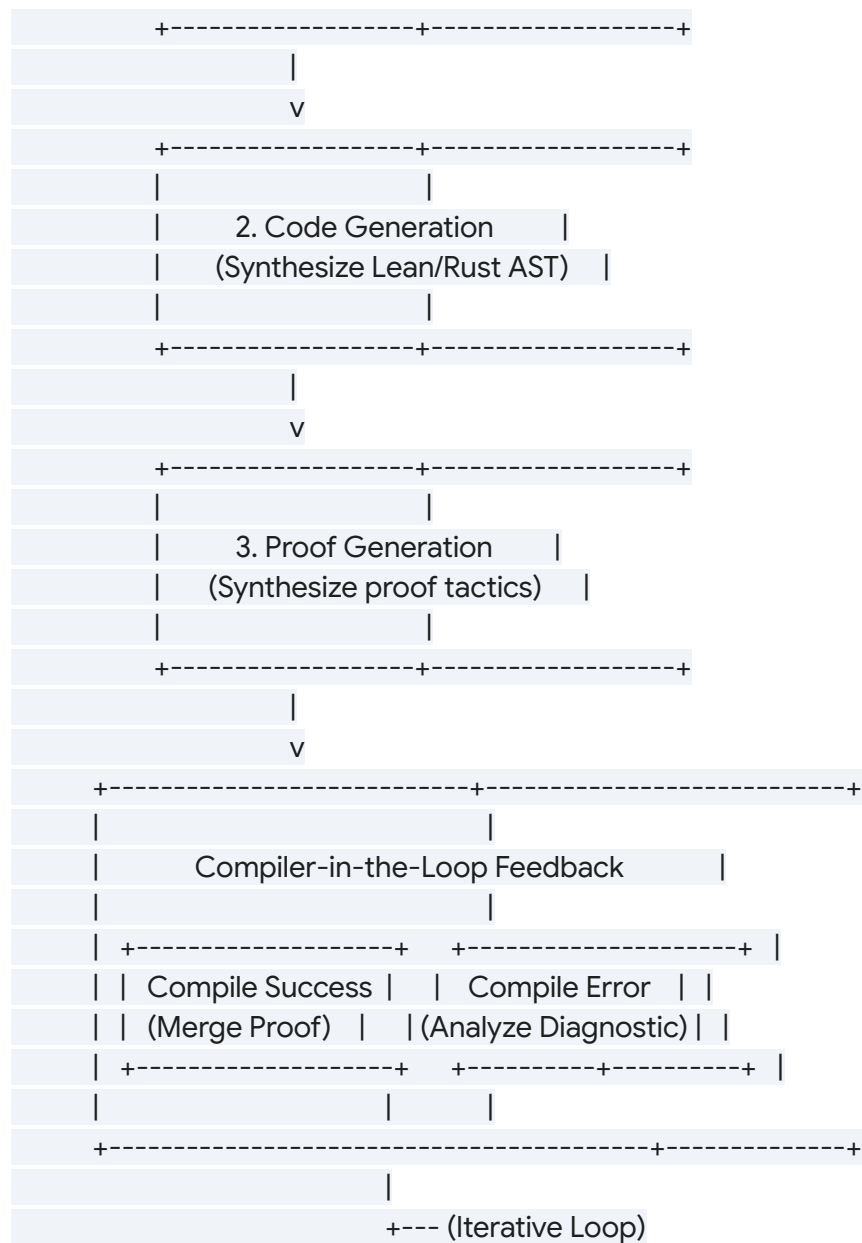
Compiler-in-the-Loop Agentic Verification Workflows

Developing manual formal proofs is traditionally the primary bottleneck in software verification, requiring significant human proof engineering. ¹⁷ To address this challenge, verification libraries can implement *agentic workflows* that automate code generation, specification, and proof checking. ¹⁹

Lean 4's architecture is uniquely suited to agentic automation. ¹³ Because Lean uses the same language for implementations, specifications, and proofs, and features a small trusted kernel and an extensible interactive Language Server Protocol (LSP), automated agents can interact directly with the compiler. ¹³ These agentic loops operate as scheduled continuous integration workflows or interactive editor assistants. ¹⁹

The agentic pipeline is structured into three distinct, iterative stages ²⁰:





During this process, the agent utilizes a "compiler-in-the-loop" paradigm.²⁰ Instead of generating code and proofs in a vacuum, the agent connects directly to the Lean LSP via the Model Context Protocol (MCP) using tools like lean-lsp-mcp.²⁰

The compiler acts as the ground truth.²⁰ The agent attempts to solve verification goals by emitting proof scripts, and then reads the feedback provided by the Lean compiler, such as type mismatches, unresolved goals, and diagnostic warnings.²⁰

Using commands like /fill-sorry, the agent targets and resolves specific unproven subgoals, refining its proofs iteratively until the compiler confirms that the proof is sound and all placeholders have been resolved.¹⁹ Autonomous workflows like Lean Squad have

demonstrated the viability of this approach by progressively verifying systems-level properties in production codebases, including Cloudflare's quiche and raft-rs.¹⁹

Proof Automation and High-Performance Symbolic Simulation

To prevent combinatorial explosions during agentic proof searches, the verification library must provide highly automated tactics that keep the agent's search space small.¹³ Lean 4's SMT-inspired grind tactic provides a powerful foundation by unifying congruence closure, E-matching, and linear arithmetic into a single automated step.¹³

However, standard term simplification strategies can suffer from performance bottlenecks when evaluating large, monadic, symbolic execution paths.²² To resolve this, Lean 4 integrates a specialized symbolic simulation framework built on the SymM monad and the SimpM simplifier.²² SymM optimizes symbolic simulation through several performance-oriented mechanisms²²:

- **Hash-Consing via AlphaShareCommon:** SymM manages a mutable state to enforce maximal sharing of subterms in memory.²⁴ This allows the simplifier to perform pointer-based cache key lookups in $O(1)$ time rather than recursive structural comparisons in $O(n)$ time, preventing hash collisions on large execution terms.²²
- **Maximal Free Variable Cache (maxFVar):** By mapping expressions to their maximal free variables, SymM checks local context compatibility for metavariables in $O(1)$ time, bypassing expensive linear context sweeps.²⁴
- **Pointer-Equality Caching:** Type inference is a well-known bottleneck in term simplification.²⁴ Caching results for type inference (inferType) and universe levels (getLevel) using pointer keys eliminates redundant typing calculations.²⁴
- **Non-contextual Branching:** Standard simplifiers add hypotheses when entering conditional branches, invalidating caches and causing exponential $O(2^n)$ behavior.²² SimpM simplifies only the condition; if the condition cannot be reduced to a boolean constant, the branch is kept symbolic, allowing the high-level simulator to handle case-splitting dynamically.²²

Comparative Architectural Analysis

When designing a verifier using Lean for Rust programs, it is critical to compare different structural verification strategies to determine the optimal design tradeoffs for an ACM-level research contribution.²⁵

Strategy	Underpinning Logic	Translation Engine	Support for Unsafe	Proof Style	Primary Strengths	Major Limitations
Deep AST Embedding in Lean 4 ¹³	Relational big/small-step operational semantics. ¹³	Monadic interpreter with fuel pattern in Lean. ¹³	High; can model raw pointer semantics explicitly. ^{2, 7}	Extrinsic; proofs over syntax tree variables. ¹³	Full control over language behavior and semantics. ¹³	High manual proof overhead; requires verifying interpreter correctness. ¹³
Shallow Monadic Translation ¹	Pure lambda calculus, state-error monads, Dijkstra monads. ¹	External compiler (Charon) to pure Lean code. ¹	Low; limited to safe Rust; requires separation logic boundaries. ²	Extrinsic; proofs over translated monadic functions. ¹	Lightweight proofs; eliminates memory reasoning entirely. ¹	Cannot directly translate raw pointer manipulation. ²
Deductive Verification ¹⁷	First-order logic, prophetic assertions, SMT theories. ^{1, 7}	Inline specifications translated to SMT-LIB via Why3/Viper. ⁵	Moderate; verified via ghost state and tracked resources. ¹⁷	Intrinsic; contracts co-mingled with Rust source code. ¹⁷	High automation; SMT solvers discharge proof goals. ¹⁷	"SMT cliff" timeouts; specification clutter in source. ¹⁷
Bounded Model Checking ²⁵	Bounded symbolic state exploration, SAT/SMT. ^{2, 5}	LLVM-IR analysis via state unrolling. ^{3, 2}	High; handles raw pointers and concurrency up to bounds. ²⁵	Harness-based; assertions written in Rust. ¹⁷	Push-button automation; finds memory bugs easily. ²⁵	Cannot prove functional correctness on unbounded

						loops. ²⁵
Agentic Compiler -in-the-Loop ¹⁹	Inductive propositions and interactive Lean tactics. ¹³	LLM agent driving Lean LSP and proof tactics. ¹⁹	Dependent on underlying translation engine. ²⁰	Hybrid; agent synthesizes extrinsic specs and proofs. ¹⁹	Near-zero manual proof labor; rapid proof convergence. ¹⁹	Hallucinations; highly dependent on Lean tactic performance. ¹³

Open Theoretical and Engineering Challenges

Translating real-world Rust into Lean 4 models introduces several open research challenges:

Loop Semantics and Termination

Pure functional languages require all functions to be proven terminating to maintain logical consistency.¹³ However, Rust programs contain arbitrary, potentially non-terminating loops and recursive operations.¹³ To address this, translation tools generate loop summaries using the "fuel" pattern.¹³ A natural number f is passed to the loop; each iteration decrements f , and the loop aborts returning Panic or None if $f = 0$.¹³ Proving correctness then requires showing that under a sufficiently large but unspecified quantity of fuel, the functional model computes the correct output.¹³ Alternatively, researchers must prove loop termination by identifying inductive invariants or well-founded relations, a task that remains challenging to automate.³

The Safe-Unsafe Frontier

Crucial systems-level components (such as memory allocators, concurrent locks, and high-performance collection types like Vec or HashMap) are implemented using unsafe Rust to bypass compiler restrictions.² Because unsafe code manipulates raw pointers, it violates the aliasing guarantees that make the forward-backward functional translation sound.²

To scale the verifier, researchers are developing hybrid verification architectures.² Under this paradigm, a stateful separation logic framework (such as Iris, mechanized in Coq or Lean) is used to verify the low-level, stateful implementation of the unsafe abstraction.² The unsafe implementation is proven to satisfy a clean, high-level functional specification.² This verified specification is then exported to the pure translator as a safe, uninterpreted functional axiom.² Thus, the rest of the application can be translated into pure, lightweight Lean models, using the verified axioms to interact with the underlying stateful abstractions.²

Mechanized Soundness of Symbolic Translation

A major gap in current Rust-to-Lean verifiers is the absence of a verified soundness proof for the translation engine itself.³⁶ Because the symbolic execution engine inside translators like Aeneas effectively acts as a borrow checker (proving that a translated program is memory-safe), formalizing this proof is a non-trivial challenge.² Proving this translation sound requires constructing a semantic model of the intermediate borrow calculus (such as LLBC) in Lean and showing that evaluated expressions preserve operational equivalence with the source program.²⁷

Publication Roadmap and Research Recommendations

For a submission targeting an ACM venue (such as POPL, PLDI, or CAV), a research project should combine theoretical design with practical evaluation.¹ A highly competitive paper can be structured around the following four-phase contribution roadmap.

Part 1: Theoretical Framework

Define a core, formal target language in Lean 4 utilizing a shallow monadic embedding with Dijkstra monads.¹² Formally specify the forward-backward translation semantics for a subset of LLBC containing mutable references and structured loops.³ Prove the soundness of this translation with respect to a simplified operational model of Rust’s memory behavior.²⁷

Part 2: System Architecture and Implementation

Build a Lean 4 library that integrates with Charon to ingest LLBC ASTs and emit type-correct monadic Lean code.¹ Leverage Lean 4’s SymM monad to implement a high-performance, pointer-equality-keyed symbolic simulator.²³ Write custom, automated tactics using the new grind attribute system to automatically evaluate and simplify monadic state updates.¹³

Part 3: Compiler-in-the-Loop Agentic Integration

Design an LLM-based verification agent that uses the lean-lsp-mcp protocol to verify the translated monadic code.¹⁹ Implement an iterative SPECGEN-CODEGEN-PROOFGEN loop where the agent utilizes compiler diagnostics and LSP goal states to synthesize proofs, automatically resolving unproven subgoals.¹⁹ Show that this automated agentic loop achieves high verification success rates while drastically reducing human proof labor.¹⁹

Part 4: Case Studies and Evaluation

Evaluate the verification pipeline on a suite of real-world, high-assurance Rust libraries.¹ Benchmark the execution time of the symbolic simulator and compare proof-to-code ratios against state-of-the-art tools like Verus, Creusot, and Kani.¹⁷ Discuss performance results and quantify how many goals are closed automatically by the compiler-in-the-loop agent versus

human engineering.¹⁹

Works cited

1. Aeneas: Bridging Rust to Lean for Formal Verification, accessed on May 19, 2026, <https://lean-lang.org/use-cases/aeneas/>
2. Aeneas | Son HO, accessed on May 19, 2026, <https://www.sonho.fr/assets/documents/aeneas.html>
3. (PDF) Aeneas: Rust verification by functional translation - ResearchGate, accessed on May 19, 2026, https://www.researchgate.net/publication/363162155_Aeneas_Rust_verification_by_functional_translation
4. Functional translation of Rust programs - Sidney Congard's website, accessed on May 19, 2026, https://dwarfobserver.github.io/assets/Internship_Rust.pdf
5. Thrust: A Prophecy-based Refinement Type System for Rust - Tohoku University, accessed on May 19, 2026, <https://www.riec.tohoku.ac.jp/~unno/papers/pldi2025.pdf>
6. arXiv:2206.07185v2 [cs.PL] 28 Sep 2022, accessed on May 19, 2026, <https://arxiv.org/pdf/2206.07185>
7. Charon: An Analysis Framework for Rust - arXiv, accessed on May 19, 2026, <https://arxiv.org/html/2410.18042v1>
8. Charon: An Analysis Framework for Rust - arXiv, accessed on May 19, 2026, <https://arxiv.org/html/2410.18042v2>
9. (PDF) Charon: An Analysis Framework for Rust - ResearchGate, accessed on May 19, 2026, https://www.researchgate.net/publication/385177625_Charon_An_Analysis_Framework_for_Rust
10. charon/docs/usage.md at main · AeneasVerif/charon - GitHub, accessed on May 19, 2026, <https://github.com/AeneasVerif/charon/blob/main/docs/usage.md>
11. Projects - Aeneas, a Verification Framework for Rust, accessed on May 19, 2026, <https://aeneasverif.github.io/projects/>
12. Foundational Multi-Modal Program Verifiers - Ilya Sergey, accessed on May 19, 2026, <https://ilyasergey.net/assets/pdf/papers/loom-preprint.pdf>
13. The Lean Programming Language and Theorem Prover - Leonardo de Moura, accessed on May 19, 2026, <https://leodemoura.github.io/static/etaps2026/>
14. DSLean: A Framework for Type-Correct Interoperability Between Lean 4 and External DSLs - arXiv, accessed on May 19, 2026, <https://arxiv.org/html/2602.18657v2>
15. Verification of Rust Cryptographic Primitives with Aeneas - CREACH LABS, accessed on May 19, 2026, https://www.creachlabs.fr/sites/default/files/public/media/document/2026-02/2026_01_fromherz.pdf
16. The Aeneas Ecosystem: Formal Verification of Rust Programs, accessed on May 19, 2026, <https://ilm.uqam.ca/en/colloquium/2025-04-30-aymeric-fromherz-aeneas/aeneas->

[uqam-apr2025.en.pdf](#)

17. Formal proofs in the Rust language - Reddit, accessed on May 19, 2026, https://www.reddit.com/r/rust/comments/1qwoe4m/formal_proofs_in_the_rust_language/
18. Applying Modern Verification Techniques to a Root-of-Trust Bootloader - Barkhausen Institut, accessed on May 19, 2026, https://www.barkhauseninstitut.org/fileadmin/user_upload/Publikationen/2025/202509_Gordon_PLOS_VerusBoot.pdf
19. Lean Squad: Exploring Automated Software Verification with Near-Zero Human Labour – @dsyme on software, accessed on May 19, 2026, <https://dsyme.net/2026/04/20/lean-squad-automated-software-verification-with-near-zero-human-labour/>
20. AGENTIC PROVING FOR PROGRAM VERIFICATION - OpenReview, accessed on May 19, 2026, <https://openreview.net/pdf?id=wpQ6rTpGY6>
21. Leanstral: Open-source agent for trustworthy coding and formal proof engineering | Hacker News, accessed on May 19, 2026, <https://news.ycombinator.com/item?id=47404796>
22. Lean.Meta.Sym.Simp.SimpM, accessed on May 19, 2026, <https://lean-lang.org/doc/api/Lean/Meta/Sym/Simp/SimpM.html>
23. Lean 4.28.0 (2026-02-17), accessed on May 19, 2026, <https://lean-lang.org/doc/reference/latest/releases/v4.28.0/>
24. Lean.Meta.Sym.SymM, accessed on May 19, 2026, <https://lean-lang.org/doc/api/Lean/Meta/Sym/SymM.html>
25. Survey tools suitability for Std safety verification - Rust Project Goals - GitHub Pages, accessed on May 19, 2026, <https://rust-lang.github.io/rust-project-goals/2024h2/std-verification.html>
26. Verify the Safety of the Rust Standard Library | AWS Open Source Blog, accessed on May 19, 2026, <https://aws.amazon.com/blogs/opensource/verify-the-safety-of-the-rust-standard-library/>
27. Towards a formal and executable specification of Rust semantics, accessed on May 19, 2026, <https://ethz.ch/content/dam/ethz/special-interest/infk/inst-pls/plf-dam/documents/StudentProjects/MasterTheses/2023-Rudi-Thesis.pdf>
28. I wrote my Master's thesis about Rust verification, exploring type invariants and ghost code, accessed on May 19, 2026, https://www.reddit.com/r/rust/comments/1ainrpg/i_wrote_my_masters_thesis_about_rust_verification/
29. A hybrid approach to semi-automated Rust verification - arXiv, accessed on May 19, 2026, <https://arxiv.org/html/2403.15122v1>
30. Formal Specification and Verification Techniques for Mutable References and Advanced Aliasing in Rust - UBC Library Open Collections, accessed on May 19, 2026, <https://open.library.ubc.ca/media/stream/pdf/24/1.0438326/4>
31. Safety+mathematical proof : r/rust - Reddit, accessed on May 19, 2026, https://www.reddit.com/r/rust/comments/1p5l70n/safetymathematical_proof/

32. A Tale of Two Case Studies: A Unified Exploration of Rust Verification with SEABMC - repositUM, accessed on May 19, 2026,
<https://repositum.tuwien.at/bitstream/20.500.12708/219563/1/Tafese%20Joseph%20-%202025%20-%20A%20Tale%20of%20Two%20Case%20Studies%20A%20Unified%20Exploration%20of%20Rust...pdf>
33. How Open Source Projects are Using Kani to Write Better Software in Rust - AWS, accessed on May 19, 2026,
<https://aws.amazon.com/blogs/opensource/how-open-source-projects-are-using-kani-to-write-better-software-in-rust/>
34. Place Capability Graphs: A General-Purpose Model of Rust's Ownership and Borrowing Guarantees - arXiv, accessed on May 19, 2026,
<https://arxiv.org/html/2503.21691v5>
35. Formalizing Rust - ralfj.de, accessed on May 19, 2026,
<https://www.ralfj.de/blog/2015/10/12/formalizing-rust.html>
36. Internship proposal: Soundness of Rust Verification in the Presence of Safe and Unsafe Code - Cambium, accessed on May 19, 2026,
<https://cambium.inria.fr/~yzakowsk/misc/internship-aeneas.pdf>
37. AeneasVerif/aeneas: A verification toolchain for Rust programs - GitHub, accessed on May 19, 2026, <https://github.com/AeneasVerif/aeneas>